

ELI — Complete Findings and Corrections (deep-read session, 2026-06-07)

Contents

A. Authoritative numbers (corrected)	1
B. Corrections — things stated wrong or undersold earlier, now accurate	2
C. Verified subsystem facts (the accurate picture)	3
D. Fixes shipped this session (committed, tested)	4
E. Honest limits (unchanged by any of the above)	5
F. The accurate one-line summary	5

□ **Current status (2026-06-28).** This doc is the 2026-06-07 deep-read record; its numbers are point-in-time. For the authoritative current state see [state_snapshot.md](#). As of 2026-06-28: **142,325 LOC / 369 .py files** under eli/ (+ the FastAPI web app api/server.py), **208** capabilities, **174** actions, full suite **7,019 passed / 42 skipped / 2 xfailed**. Major additions since this read: a self-hosted **web app + dashboard**, **ELI's own MQTT smart-home** (Home Assistant removed), **RBAC** + a **tamper-evident HMAC'd audit trail**, research collaboration, browser voice, born-locked secret files, and a **monitored Internet toggle**.

What this is. A factual, unbiased record of everything established about ELI during a full deep-read session: the corrections to earlier (wrong or undersold) statements, the verified facts about every major subsystem, and the fixes shipped. No salesmanship — just what the code does. Companion to capability_catalogue.md (the exhaustive action/module map), what_eli_is.md (the human portrait), and project_overview.md (engineering verdict).

Method. Built from reading the algorithm-bearing bodies of every major subsystem plus the full structural skeleton of all 364 files. The three giant files (engine 13.4k / executor 14.3k / GUI 11.0k) were read at behavioural + structural level (entry flow, dispatch, handlers, tabs), not literally line-by-line; that residue is accretion (e.g. 51 PHASE... patch markers in engine.py), not undocumented capability.

A. Authoritative numbers (corrected)

Fact	Earlier stated	Verified
Lines of code	116,958 / ~122k	139,867
Python files	315 / 330	364
Test files	—	166
Capabilities (manifest, measured by capability_sync)	"~174 actions"	208 (183 routable: ~170 executor/router + 13 plugin) — ~ 110 distinct after alias families
Bus agents	"15"	14
except Exception blocks	2,322	~2,565
Commits (start of session)	—	~225 → 231+ this session
Running model	—	openhermes-2.5-mistral-7b.Q3_K_M (but model-agnostic)

B. Corrections — things stated wrong or undersold earlier, now accurate

1. **Agent count.** It is **14** specialist agents in `_ALL_AGENTS` (memory, system, habit, self_improvement, proactive, frontier, plugin, capability, voice, orchestrator, file_code, reflection, introspection, knowledge_graph) + the `AgentOrchestrator` that sequences them. Earlier “15” was wrong.
2. **world/agency — NOT scaffolding.** It is a real **embodied self-state model** (`autonomy_engine.py`): an awareness vector (memory_confidence, reflection_depth, repair_pressure, evidence_confidence, uncertainty, autonomy_pressure, focus, curiosity, tool_activity) updated with **exponential easing + time decay**; an avatar that moves through cognitive “rooms” (Reflection Chamber, Anomaly Room, Evidence Wall, Simulation Lab, ...) mapped to actual reasoning stages; with provenance IDs, periodic snapshots, a journal, and permission-classed actions.
3. **Autonomy — NOT thin.** A governed **mission/goal layer** exists: `goal_store/goal_models` (goals carry priority, cadence, risk, constraints, success_criteria, autonomy_mode) → `autonomy_scheduler` (policy modes `observe_only/proposal_only/goal_driven`, cooldowns) → `goal_tick` emits **governed proposals into a queue for human approval** (not free-running actions) → `attention_queue` + the GUI operator console. Supervised autonomy, safety-gated by default.
4. **Self-healing environment repair — entirely missed earlier.** `grounded_remediation.py` (1,604L): on a failed action (e.g. `OPEN_APP` for an uninstalled app), `capture_executor_failure` triggers a **diagnosis → repair plan** (apt/snap/flatpak install candidates) → **offer** → on confirmation, **executes** it (sudo terminal when needed, package-lock contention handling, post-install verification). Linux-only. ELI can literally install a missing app for you.
5. **News — a rolling intelligence cadence, not “fetch news”.** `news_synthesis.py`: every 3 hours it LLM-summarises that window’s articles into a stored “news reflection”; the morning report consolidates the eight 3-hour reflections from 24h into one digest; it derives the user’s interest terms from memory and builds an interest-matched block, spread across domains.
6. **Reasoning modes are REAL multi-pass algorithms, not prompt labels.** In `engine.py`: `_run_self_consistency` (N samples + a consensus-selection pass, n+1 stages), `_run_tree_of_thoughts` (k branches explored/pruned), `_run_constitutional_ai` (draft → principle-critique → revise), `_run_chain_of_thought`, and `quick`. All private (reasoning leaks stripped).
7. **Adaptivity — undocumented earlier.** `engagement_tracker.py` scores each turn’s depth, keeps a recency-weighted session score, and **auto-escalates the reasoning mode** (quick→chain_of_thought→self) as a conversation deepens, with a current-turn floor so a single deep question escalates immediately. `tone_analyzer.py` separately learns tone preferences.
8. **Voice — a full pipeline, not “STT + wake gate”.** `audio_stt.py` (1,848L): wake-word arming with debounce + a guarded window, **incomplete-command handling** (“open” → waits for the target/service), **self-echo suppression** (won’t transcribe ELI’s own TTS back as a command), **output ducking** (drops media/system volume so it can hear you), safe-direct vs guarded-command classification, and a **per-user voice profile** that biases recognition.
9. **Self-improvement — a full cycle, not a single patch.** `run_patch_cycle` clusters recurring failures (≥2 occurrences) → generates a fix per failure → syntax-validates → applies via `apply_code_patch` (which does an isolated import smoke-test + timestamped backup + **auto-revert on failure**) → reports per-patch outcomes; dry-run mode available.
10. **Living user profile.** `user_info_builder.py` (746L) assembles a versioned profile from semantic memory + user_patterns + stable memories + recent turns, filters profile noise, categorises into sections, hashes them, and **appends a diff whenever the profile changes**.
11. **Engine entry pipeline — two undocumented features.** `engine.process()`:
 - (a) a **prompt-injection guard** (`_eli_sanitize_user_input`) runs FIRST, before router/LLM see the raw text; (b) a **multi-question splitter** — messages with >1 ? and >25 words are split into ≤4 standalone sub-answers, plus a compound “who are you AND who am I” splitter.

12. **DATA_FABRICATOR** generates a document from a topic (via CREATE_DOCUMENT) **and opens it in an editor** — not synthetic-data generation as earlier guessed. CHECK_CHRONAL_ALIGNMENT is a time easter-egg.
 13. **Image generation — a capable procedural renderer.** tools/image_engine/: 10+ distinct procedural scene types (landscape, mountains, water, floating island, product, poster, space, city, abstract, emblem) with atmosphere, particles, post-processing, composition planning, palette/style systems, batch
 - project rendering — *no model required*; SSD-1B diffusion is the optional add-on (weights not bundled).
-

C. Verified subsystem facts (the accurate picture)

Request pipeline. Router (router_enhanced.py, ~174 actions, regex-first + LLM-intent fallback) → engine gate (quick/fast/direct → 14-agent bus fastpath; non-quick → 12-stage orchestrator) → reasoning mode → executor (167 dispatch points) → governed output. SEQUENCE chains multiple actions via _execute_impl.

Agent bus. 15 agents on a dependency DAG (core/dag.py topological layers, parallel where independent). Confidence aggregation is **weight-free and calibrated**: contribution = evidence_quality × payload-density × a *learned* per-(agent,action) calibration from the agent_metrics table; single-agent contributions capped. Per-action agent selection narrows the set.

Orchestrator. 12 stages: HyDE expansion → parallel FAISS (vector) + FTS5 (keyword) + KG multi-hop BFS + conversation + document RAG → hybrid merge → heuristic rerank (lexical × recency × importance — reranker.py is explicitly a dependency-free reranker, NOT yet a neural cross-encoder; that's the designed upgrade point) → precise context assembly. Mode-aware retrieval budgets.

Memory. Four SQLite stores — **user.sqlite3** (conversations, memories, KG, news, habits, user_patterns), **agent.sqlite3** (agent/self-improvement: dispatches, metrics, code_patches, failures, improvements — now the *single* failure store), **system_index.sqlite3** (your machine's executables for app launch — thousands, indexed live per machine), **coding_memory.sqlite3** (bug→fix memory). Plus FAISS (L2, 1/(1+dist) similarity, nomic embedder, keyword fallback), a knowledge graph (multi-hop BFS), working memory (turn-scoped pins), and **dynamic facts** (volatile project/interest facts age out unless reaffirmed).

Coding agent (eli/coding/). DAG subtask decomposition → **UCB1 tree search** over a temperature-ladder beam → **3-gate verification ladder** (syntax → sandboxed execution → synthesised tests) → weighted scoring (tests dominate; broken syntax hard-capped at 0.05) → **(bug→fix) long-term memory** feeding the repair loop → cost-based foreground/background decision.

Grounding / self-honesty. deterministic_grounding_gate.py (4,292L) renders control/status/identity answers **from live measurement, bypassing the model**; grounding_escalation.py escalates poorly-grounded factual questions through agent tiers instead of letting the model confabulate; control_contracts.py validates control output against gathered evidence; evidence_ledger/ evidence_arbitration record and score evidence; capability_sync AST-measures the capability manifest (so the 208/183 count is measured, not asserted).

Security. Offline-by-default at the **socket** (netguard, fail-closed) + fail-closed command/path/app allowlist (SecurityManager) + prompt-injection guard at pipeline entry + AST-validated, hash-trusted custom agents + crisis-guard persona steering on self-harm signals.

Inference / hardware. GGUF via llama.cpp, model-agnostic (chat template auto-detected by family); hardware profiler auto-fits ctx/gpu_layers/batch with adaptive fallback (observed cascading through 6 configs to fit 8 GB VRAM); a user-tunable synthesis prompt cap (ELI_SYNTH_MAX_PROMPT_CHARS).

Learning. Real LoRA fine-tuning (lora_trainer.py — torch/PEFT/transformers Trainer), datasets built from the user's own conversations with PII redaction; human-gated, operator-invoked, separate Phi-3 base.

Perception. faster-whisper STT (full voice pipeline, §B.8), Piper TTS with unspeakable-fragment guard, local GGUF vision (Moondream fast + primary, hot-swap, CPU-pinned CLIP), OCR screen-element location, MediaPipe gaze (+ calibration + One-Euro smoothing @10Hz), cross-platform OS control.

Proactivity. `proactive_daemon.py` 10-minute loop: pattern + code analysis, habit detection + *offer* (never silent activation), morning report, error tracking. The habit scheduler runs active app-launch habits and self-heals legacy malformed rows.

GUI. PySide6, 12 top-level tabs (Chat, Habits, Images, Self-Improve, Proactive [Suggestions/Summaries/Insights/Ha Improve/Memory sub-tabs], Quick-Actions [drag-drop action board], Screen, IDE, Eli's World, Labs, Coding, Files, Settings) + operator console + proactive dock + first-boot wizard.

Extensibility. 11 built-in plugins + manager (install/enable/disable, builtin stubs); custom agents (AST-validated → hash-trusted → live-registered); `capability_sync` keeps the manifest in lockstep with the code.

D. Fixes shipped this session (committed, tested)

Area	Change
Agent knowledge-gathering	<code>file_code</code> searches the whole repo; memory agent multi-hop deepen; <code>capability/voice</code> triggers broadened
Model degeneration (-/-G)	Context-bloat synthesis quality cap (<code>ELI_SYNTH_MAX_PROMPT_CHARS</code> , head+tail preserving); then gather limits raised
User control	New core/cognition_tunables.py + GUI
Web	Cognition tab exposing every gather limit
Routing	Confirmed toggle-gated first-class (netguard fail-closed when off)
Plugins	Action-synonym normalisation (<code>NEWS_SEARCH</code> → <code>NEWS_FETCH</code> , <code>DAILY/WEEKLY_REPORT</code> → <code>MORNING_REPORT</code> , <code>WEBSITE_SEARCH</code> → <code>WEB_SEARCH</code>)
Habits	Fixed Plugin manager unavailable: name 'log' is not defined (missing module logger)
Diagnostics	Scheduler now runs active app-launch habits (over-broad guard narrowed) + once-per-minute dedupe; self-heals legacy 00:00/bare-token rows
Failures	<code>RUNTIME_AUDIT</code> gained live health probes (plugin mgr, memory, agent bus, habit integrity, recent failures)
Governance	Consolidated to ONE store (<code>agent.sqlite3</code>); <code>mark_failure_resolved</code> + status-filtered reads
GUI	3 overlapping modules → canonical <code>output_governor</code> + shims; <code>normalize_response</code> signature collision fixed → <code>clean_gguf_artifacts</code>
Eval	Folder drag-drop inserts a bare path (not a [File:] wrapper)
Grounding gate	Runs under pytest (<code>tests/test_eval_cases.py</code>) — auto with the suite
Reasoning modes	Removed one provably-dead v10 fragment, oracle-verified byte-identical Renamed Quick/Normal/Advanced/Research/Expert (public layer; internal keys kept) + per-mode agent time budgets (1.0×→2.5×)

Area	Change
Autonomous deepening	Confidence-driven iterative deepening that escalates the mode + raises gather per iteration until a per-mode target is met (quick never deepens); background deepening surfaces a better answer async for weak Quick factual turns
Docs	Refreshed all 21 blueprints; added what_eli_is.md, what_eli_can_do.md, capability_catalogue.md (+Part 6 reasoning/deepening), this doc

E. Honest limits (unchanged by any of the above)

- **The small local model is the ceiling.** Every quality issue traces to it; ~40% of the codebase exists to compensate (grounding/governance).
- **Accretion debt.** engine.py has 51 PHASE... patch markers; the grounding gate stacks 7 render_action layers; ~2,565 except Exception swallow errors into silent fallbacks (the reason bugs surface only via runtime logs).
- **God-files.** Three single files (~10–13k LOC each: executor, engine, GUI) = ~41% of the codebase; high regression surface.
- **Instrumented ≠ exercised.** The world/agency and governed-autonomy layers are coherent and wired but lightly used in daily flow; diffusion image-gen is dormant (weights not bundled).
- **Solo project.** Polish, adoption, ecosystem, and reliability lag funded competitors — the gap is consolidation and a bigger brain, not vision.

F. The accurate one-line summary

ELI is a **~133,400-line, 352-file, 100%-local, model-agnostic personal cognitive runtime**: a 14-agent calibrated bus on a 12-stage retrieval pipeline with 5 real multi-pass reasoning modes; a four-store adaptive memory; a deterministic self-honesty layer; a frontier-grade self-verifying coding agent; safe self-patching and real local LoRA self-training; self-healing app-install remediation; an embodied self-state model and governed mission-autonomy; full local voice/vision/gaze/OS-control; ~110 distinct capabilities — all private, offline by default, and owned. Its ceiling is the size of the local brain you give it; everything around that brain is already built.